0xCurvy Audit

Gnosis Ltd - Report by Côme du Crest

2026-08-31

Table of contents

- Table of contents
- 0xCurvy Audit
  - Scope
  - Context
  - Assumptions
  - Status
  - Legal Information And Disclaimer
- Issues
  - [Med] `onlyRecovery` uses `tx.origin`
  - [Low] Constrained aggregator config
  - [Low] Cannot reattempt portal shielding
  - [Info] `updateConfig()` may emit `ConfigUpdated` for non-updated config
  - [Info] Aggregator initial root not valid
  - [Info] Vault deposit gas fee collected instead of sent to relayer

0xCurvy Audit

This document presents the findings of a smart contract audit conducted by Côme du Crest.

For any inquiry contact me by email at come.ducrest@gmail.com or via telegram @ComeduCrest.

Scope

The scope includes selected contracts of 0xCurvy/contracts as of commit `0x5f6e43c`.

The selected contracts are:

- aggregator-alpha/CurvyAggregatorAlphaV6.sol
- vault/CurvyVaultV6.sol
- portal/PortalFactory.sol
- portal/Portal.sol
- utils/Types.sol

The scope also includes the latest version (v2) at the latest commit `0x36b6918`.

Context

This audit follows an audit already performed on same commit hash by `ethernal` (`Curvy 2026 Audit Report Final.pdf`).

The previous audit and the project's documentation properly explain the concept behind the project.

Assumptions

Portals are created with four parameters `ownerHash`, `exitAddress`, `exitChainId`, and `recovery`. My assumptions throughout this audit is that `ownerHash` and `recovery` are never repeated.

If both are repeated at the same time, shield portal deployment should fail as it will attempt to re-deploy a portal at an already used address via `create2`.

If `ownerHash` or `recovery` are repeated by themselves, part of the privacy should be lost as shield portal leak amount transferred.

If `ownerHash` is repeated then the `noteId` computed by `CurvyAggregatorAlphaV6` is not unique for the same `amount - fee`.

The admin and operator roles are fully trusted. Admins can update the UUPS vault and aggregator. Operators can manipulate the bridge data during portal deployment.

Status

Only one Med finding has been reported and depending on the assumptions of the protocol it may be acknowledged.

The report has been sent to the core developer.

Legal Information And Disclaimer

1. This report is based solely on the information provided by 0xCurvy (the “Company”), with the assumption that the information provided to Gnosis is authentic, accurate, complete, and not misleading as of the date of this report. Gnosis has not conducted any independent enquiries, investigations or due diligence in respect of the Company, its business or its operations.
2. Changes to the information contained in the documents, repositories and any other materials referenced in this report might affect or change the analysis and conclusions presented. Gnosis is not responsible for monitoring, nor will we be aware of, any future additions, modifications, or deletions to the audited code. As such, Gnosis does not assume any responsibility to update any information, content or data contained in this report following the date of its publication.
3. This report does not address, nor should it be interpreted as addressing, any regulatory, tax or legal matters, including but not limited to: tax treatment, tax consequences, levies, duties, data privacy, data protection laws, issues relating to the licensing of information technology, intellectual property, money laundering and countering the financing of terrorism, or any other legal restrictions or prohibitions. Gnosis disclaims any liability for omissions or errors in the findings or conclusions presented in this report.
4. The views expressed in this report are solely our views regarding the specific issues discussed within this report. This report is not intended to be exhaustive, nor should it be construed as an assurance, guarantee or warranty that the code is free from bugs, vulnerabilities, defects or deficiencies. Different use cases may carry different risks, and integration with third-party applications may introduce additional risks.
5. This report is provided for informational purposes only and should not be used as the basis for making investment or financial decisions. This report does not constitute investment research and should not be viewed as an invitation, recommendation, solicitation or offer to subscribe for or purchase any securities, investments, products or services. Gnosis is not a financial advisor, and this report does not constitute financial or investment advice.
6. The statements in this report should be considered as a whole, and no individual statement should be extracted or referenced independently.
7. This report is addressed exclusively to the Company, and except with prior written consent from Gnosis, it may not be shared, disclosed, transmitted, or relied upon by any other person or entity for any purpose.
8. To the fullest extent permitted by applicable laws, Gnosis disclaims any and all other liability, whether in contract, tort, or otherwise, that may arise from this report or the use thereof.

Issues

[Med] `onlyRecovery` uses `tx.origin`

Summary

The recovery mechanism of portals uses the `onlyRecovery` modifier that enforces `tx.origin` is the recovery address. If the recovery address interact with any other smart contract, their portals can be recovered in their name.

Vulnerability Detail

The `onlyRecovery()` modifier enforces transaction origin:

```
    modifier onlyRecovery() {
        require(tx.origin == recovery, "Portal: Only recovery");
        _;
    }

    function recover(address tokenAddress, address to) external onlyRecovery {
        ...
    }
```

Impact

If the recovery address interacts with any smart contract that can divert execution flow, their pending portals can be recovered in their names to addresses not under their control.

It seems intended that the `recovery` address is a throwaway address that never interact with anything else which is why this issue is not given `High` severity.

Code Snippets

https://github.com/0xCurvy/contracts/blob/36b691843c1fa25ae9eaa5426dff561e41df6a6e/evm/src/v2/portal/Portal.sol#L29

Recommendation

In `Portal.recover()` check that `msg.sender` is the portal factory and in the factory check that `msg.sender` is the recovery address before calling `recover()`.

Alternatively, recover the funds only to the `recovery` address to disincentivize anyone to exploit this flaw.

[Low] Constrained aggregator config

Summary

In `CurvyAggregatorAlphaV6` there are few constraints set in `updateConfig()` on the values of `maxDeposits`, `maxAggregations`, or `maxWithdrawals` while in practice very few (sometime one) values produce correct contract behaviour.

Vulnerability Detail

`updateConfig()` allows any non-zero values for `maxDeposits`, `maxAggregations`, or `maxWithdrawals`:

```
    function updateConfig(CurvyTypes.AggregatorConfigurationUpdateV2 memory
        _update) external onlyOwner returns (bool) {
        ...
        if (_update.maxDeposits != 0) {
            maxDeposits = _update.maxDeposits;
        }
        if (_update.maxAggregations != 0) {
            maxAggregations = _update.maxAggregations;
        }
        if (_update.maxWithdrawals != 0) {
            maxWithdrawals = _update.maxWithdrawals;
        }

        return true;
    }
```

If `maxDeposits` is 1 then `publicInputs[1]` will not be deleted from `_pendingIdsQueue` while it is processed by the proof verifier. If it is 3 then `publicInputs[2]` is both a note and the tree root. So it must be `2`:

```
    function commitDepositBatch(
        uint256[2] memory proof_a,
        uint256[2][2] memory proof_b,
        uint256[2] memory proof_c,
        uint256[4] memory publicInputs
    ) public {
        for (uint256 i = 0; i < maxDeposits; i += 1) {
            uint256 noteId = publicInputs[i];
            if (noteId != 0) {
                if (!_pendingIdsQueue[noteId]) revert
                    NoteNotScheduledForDeposit();
                delete _pendingIdsQueue[noteId];
            }
        }

        uint256 numPublicInputs = publicInputs.length;
```

```
16
17          if (_notesTreeRoot != publicInputs[numPublicInputs - 2]) revert
                InvalidNotesRoot();
18
19          if (!insertionVerifier.verifyProof(proof_a, proof_b, proof_c,
                publicInputs)) revert InvalidProof();
20
21          _notesTreeRoot = publicInputs[numPublicInputs - 1];
22      }
```

`maxAggregations` must be at most 5 to have enough room for the nullifiers and tree roots, it should probably be 5 for correct proof verification:

```
1       function commitAggregationBatch(
2           uint256[2] memory proof_a,
3           uint256[2][2] memory proof_b,
4           uint256[2] memory proof_c,
5           uint256[14] memory publicInputs
6       ) public {
7           uint256 oldNullifiersTreeRoot = publicInputs[2 * maxAggregations + 1];
8           uint256 newNullifiersTreeRoot = publicInputs[2 * maxAggregations + 2];
9           uint256 oldNotesTreeRoot = publicInputs[2 * maxAggregations + 3];
10          uint256 newNotesTreeRoot = publicInputs[2 * maxAggregations + 4];
11
12          if (_notesTreeRoot != oldNotesTreeRoot) revert
                CurrentNoteTreeRootMismatch();
13          if (_nullifiersTreeRoot != oldNullifiersTreeRoot) revert
                CurrentNullifierTreeRootMismatch();
14
15          if (!aggregationVerifier.verifyProof(proof_a, proof_b, proof_c,
                publicInputs)) revert InvalidProof();
16
17          // Update the roots of the trees
18          _notesTreeRoot = newNotesTreeRoot;
19          _nullifiersTreeRoot = newNullifiersTreeRoot;
20      }
```

`maxWithdrawals` must be `3` otherwise some withdrawals are not processed by the `for` loop while they were verified by the withdraw verifier:

```
1       function commitWithdrawalBatch(
2           uint256[2] memory proof_a,
3           uint256[2][2] memory proof_b,
4           uint256[2] memory proof_c,
5           uint256[9] memory publicInputs
6       ) public {
7           if (publicInputs[2] != _nullifiersTreeRoot) {
8               revert CurrentNullifierTreeRootMismatch();
9           }
10          if (publicInputs[1] != _notesTreeRoot) revert
                CurrentNoteTreeRootMismatch();
11
12          if (!withdrawVerifier.verifyProof(proof_a, proof_b, proof_c,
                publicInputs)) revert InvalidProof();
```

```
13
14          // Update the root of the nullifier tree
15          _nullifiersTreeRoot = publicInputs[0];
16
17          uint256 numPublicInputs = publicInputs.length;
18
19          // Transfer withdrawals
20          for (uint256 i = 0; i < maxWithdrawals; i += 1) {
21              uint256 amount = publicInputs[3 + i];
22              address destinationAddress = address(uint160(publicInputs[3 +
                    maxWithdrawals + i]));
23              if (amount != 0) {
24                  curvyVault.withdraw(
25                      publicInputs[numPublicInputs - 1], // tokenId
26                      destinationAddress,
27                      amount
28                  );
29              }
30          }
31      }
```

Impact

The aggregator could be misconfigured by `updateConfig()`.

Code Snippets

https://github.com/0xCurvy/contracts/blob/5f6e43cf153c0402bab7d1c041978ac8cf7a384b/contracts/aggregator-alpha/CurvyAggregatorAlphaV6.sol#L84

Recommendation

Use constants for these values instead of setting them up.

[Low] Cannot reattempt portal shielding

Summary

The function `Portal.shield()` uses the `onlyOnce` but sets the `_used` value to `false` when failing to get token address from vault in a hope to allow the function to be callable once again. However, the wrapping `onlyOnce` modifier sets `_used` to true after the call and prevents this.

Vulnerability Detail

The function `shield()` sets `_used` to false before returning to allow the function to be called again:

```
    function shield(
        CurvyTypes.Note memory note,
        address curvyAggregatorAlphaProxyAddress,
        address curvyVaultProxyAddress
    ) external onlyOnce {
        ...

        address tokenAddress;
        try curvyVault.getTokenAddress(note.token) returns (address
            _tokenAddress) {
            tokenAddress = _tokenAddress;
        } catch {
            emit ShieldingFailed(note.ownerHash, tokenAddress, note.amount, "
                Failed to get token address from vault");
            // Here we just do a return because we want the deployment to pass
                so that the user can call the recover method.
            _used = false; // We also set the used to false so that if the
                token gets registered in the near future, the user may reattempt
                 shielding.
            return;
        }
        ...
    }
```

However, the `onlyOnce()` modifier sets `_used` to `true` after execution of the function, rendering this `_used = false` pointless:

```
    modifier onlyOnce() {
        require(!_used, "SingleUse: Already used");
        _;
        _used = true;
    }
```

Impact

`shield()` cannot be called a second time when failing to get the token address from the vault.

Code Snippets

https://github.com/0xCurvy/contracts/blob/5f6e43cf153c0402bab7d1c041978ac8cf7a384b/contracts/portal/Portal.sol#L67

Recommendation

Sets `_used` to `true` before calling the wrapped function in the modifier:

```
    modifier onlyOnce() {
        require(!_used, "SingleUse: Already used");
        _used = true;
        _;
    }
```

This also prevents re-entrancy angles for `shield()` and `bridge()` functions.

[Info] `updateConfig()` may emit `ConfigUpdated` for non-updated config

Summary

The function `PortalFactory.updateConfig()` to update the contracts' config will emit an event `ConfigUpdated` for configs that have not been updated when the pointed contract has no code.

Vulnerability Detail

The function `updateConfig()` ignores contracts that have no code but still emits the events the event `ConfigUpdated` with the ignored contracts:

```
1     function updateConfig(
2         address curvyVaultProxyAddress,
3         address curvyAggregatorAlphaProxyAddress,
4         address lifiDiamondAddress
5     ) external onlyRole(AUTHORITY_ROLE) returns (bool) {
6         if (curvyVaultProxyAddress.code.length > 0) {
7             _curvyVaultProxyAddress = curvyVaultProxyAddress;
8         }
9
10        if (curvyAggregatorAlphaProxyAddress.code.length > 0) {
11            _curvyAggregatorAlphaProxyAddress =
                  curvyAggregatorAlphaProxyAddress;
12        }
13
14        if (lifiDiamondAddress.code.length > 0) {
15            _lifiDiamondAddress = lifiDiamondAddress;
16        }
17
18        emit ConfigUpdated(_curvyVaultProxyAddress,
              _curvyAggregatorAlphaProxyAddress, _lifiDiamondAddress);
19
20        return true;
21    }
```

Impact

Reading `ConfigUpdated` events emitted from the Portal Factory may lead to a different config than what is actually applied in the contract.

Code Snippets

https://github.com/0xCurvy/contracts/blob/36b691843c1fa25ae9eaa5426dff561e41df6a6e/evm/src/v2/portal/PortalFactory.sol#L82

Recommendation

Revert if no code is found at the input address.

[Info] Aggregator initial root not valid

Summary

The function `CurvyAggregatorAlphaV2.initialize()` sets an initial root value for the current notes but does not add this valid to the mapping `validNotesRoot`.

Vulnerability Detail

The initial notes tree root is not added to `validNotesRoot`:

```
1      function initialize(address initialOwner) public initializer {
2          ...
3
4          currentNotesTreeRoot =
               411468604756416044961160361541856745700810155509070353540589165626265864446 3;
5      }
```

Impact

Requests that need to use this root will not be processable.

Code Snippets

https://github.com/0xCurvy/contracts/blob/36b691843c1fa25ae9eaa5426dff561e41df6a6e/evm/src/v2/aggregator-alpha/CurvyAggregatorAlphaV2.sol#L107

Recommendation

Add `validNotesRoot[currentNotesTreeRoot] = true;` in the initialize function.

[Info] Vault deposit gas fee collected instead of sent to relayer

Summary

The gas fee computed during deposit into the vault is accrued to `_feeCollectorAddress` and not the relayer as in `withdraw()`.

Vulnerability Detail

The gas fees are accrued to `_feeCollectorAddress` in `deposit()`:

```
    function deposit(address tokenAddress, address to, uint256 amount) public
        payable onlyCurvyAggregator() {
        ...

        PackedGasFees storage tokenGasFees = _perTokenGasFees[tokenId];
        // Both costs live in the same packed slot, so this is a single SLOAD.
        uint256 gasFees = uint256(tokenGasFees.pendingNoteCommitment) +
            tokenGasFees.portalDeployment;

        uint256 depositedAmount = amount - gasFees;
        if (depositFee != 0) {
            uint256 feeAmount = (amount * depositFee) / FEE_DENOMINATOR;
            depositedAmount -= feeAmount;

            _balances[to][tokenId] += depositedAmount;
            _balances[_feeCollectorAddress][tokenId] += feeAmount + gasFees;
        } else {
            _balances[to][tokenId] += depositedAmount;
            _balances[_feeCollectorAddress][tokenId] +=  gasFees;
        }
        ...
    }
```

While in `withdraw()` fees are sent to the input `gasFeeRecipient`:

```
    function withdraw(
        uint256 tokenId,
        address to,
        uint256 amount,
        address gasFeeRecipient
    ) external onlyCurvyAggregator {
        ...
        uint256 gasFee = _perTokenGasFees[tokenId].withdrawal;
        amountAfterFees -= gasFee;

        // Withdraw the net to the destination and the gas reimbursement to the
            relayer.
        if (tokenId != ETH_ID) {
            // We are withdrawing ERC20s
            IERC20(tokenAddress).safeTransfer(to, amountAfterFees);
```

```
15              if (gasFee > 0) IERC20(tokenAddress).safeTransfer(gasFeeRecipient,
                    gasFee);
16          } else {
17              // We are withdrawing ETH
18              (bool success,) = to.call{value: amountAfterFees}("");
19              if (!success) revert ETHTransferFailed();
20              if (gasFee > 0) {
21                  (bool gasSuccess,) = gasFeeRecipient.call{value: gasFee}("");
22                  if (!gasSuccess) revert ETHTransferFailed();
23              }
24          }
25          ...
26      }
```

Impact

This may be intended as the `msg.sender` in case of deposit is a protocol operator. Otherwise the fee are not distributed to the correct address.

Code Snippets

https://github.com/0xCurvy/contracts/blob/36b691843c1fa25ae9eaa5426dff561e41df6a6e/evm/src/v2/vault/CurvyVaultV2.sol#L315

Recommendation

Acknowledge that this is intended behaviour or update fee distribution.